MINI PROJECT · OOP FUNDAMENTALS

BankVault

A simple bank account console app

C# · .NET 10 · System.Text.Json · xUnit

```
==========================
BANK VAULT SYSTEM
==========================
1. Deposit
2. Withdraw
3. Transfer
4. View Balance
5. Transaction History
0. Exit
```

THE IDEA

What does BankVault do?

A console app that manages bank accounts and keeps the data between runs.

01

Deposit

Add money to an account.

02

Withdraw

Take money out, only if the balance is enough.

03

Transfer

Move money from one account to another.

04

View balance

See the current balance at any time.

05

History

Every operation is logged with time and amount.

06

Saved data

Accounts are stored in a JSON file between runs.

WHAT WE PRACTICED

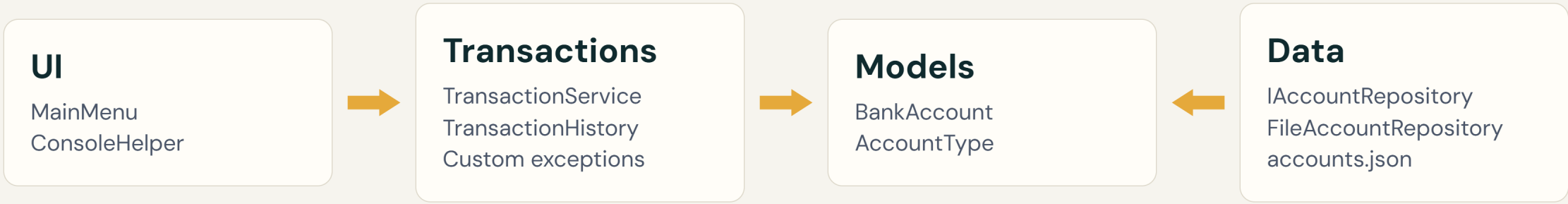
OOP concepts in the project

Concept	Where you can see it
Classes and objects	BankAccount, TransactionService, MainMenu
Encapsulation	Balance has a private set: only Deposit and Withdraw change it
Constructors	BankAccount validates its data when it is created
Custom exceptions	InvalidAmountException, InsufficientFundsException
Interfaces	IAccountRepository, implemented by FileAccountRepository
Separation of concerns	Models, Transactions, UI and Data live in separate folders

DESIGN

Four layers, one job each

Program.cs connects all the layers together



Each layer is independent, so every part can be built and tested on its own.

LAYER 1 · MODELS

BankAccount protects its balance

Properties

- **AccountNumber** and **OwnerName**
- **Balance**, with a private set
- **Type**: Savings or Current
- **OpenedDate**

The constructor rejects an empty number, an empty name or a negative starting balance.

```
public void Withdraw(decimal amount)
{
  if (amount ⩽ 0)
    throw new ArgumentException(...);
  if (amount > Balance)
    throw new InvalidOperationException(...);
  Balance -= amount;
}
```

LAYER 2 · TRANSACTIONS

TransactionService runs every operation

Step 1

Validate

ValidateAmount rejects zero or a negative amount.

Step 2

Check funds

Withdraw and Transfer need enough balance.

Step 3

Update

BankAccount changes its own balance.

Step 4

Log

TransactionHistory stores time, type and amount.

InvalidAmountException

Thrown when the amount is zero or negative.

InsufficientFundsException

Thrown when the amount is bigger than the balance.

LAYER 3 · DATA

Accounts are saved to a JSON file

IAccountRepository

The contract: what any storage must be able to do.

- Add, Update, Delete
- GetByAccountNumber, GetAll
- SaveChanges, LoadData

FileAccountRepository implements it with System.Text.Json.

```
// data/accounts.json
[
  {
    "AccountNumber": "1002",
    "OwnerName": "Ahmed",
    "Balance": 1250,
    "Type": "Savings"
  }
]
```

LAYER 4 · UI

A simple menu that never crashes

```
==========================
BANK VAULT SYSTEM
==========================
1. Deposit
2. Withdraw
3. Transfer
4. View Balance
5. Transaction History
0. Exit
Enter your choice: _
```

MainMenu

A switch statement inside a loop. It runs until the user picks 0.

ConsoleHelper

ReadInt and ReadAmount use TryParse and ask again when the input is not a number.

Friendly errors

Exceptions are caught and shown as a clear message instead of closing the app.

HOW IT RUNS

What Program.cs does

1

Load

Read the saved accounts from the JSON file.

2

Sample

If there are none, create a sample account.

3

Pick

Use the first account in the menu.

4

Run

Create the service and start the menu.

5

Save

On exit, write the new balance to the file.

Run it with `dotnet run --project src/BankVault.csproj`